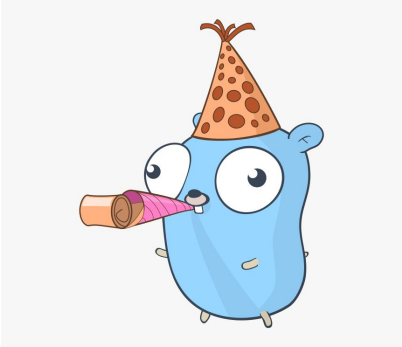
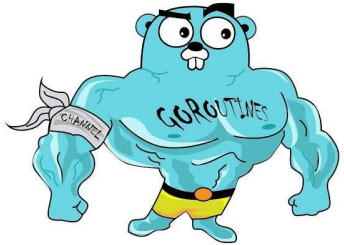
How Far Can We Go?

Coffee?



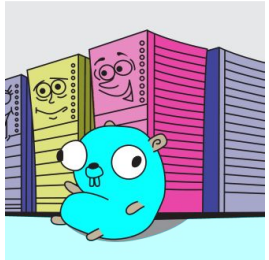

CHANNEL
GOROUTINES

Future
of Fast
Code Achieved
Through PGO!

marshmallow

GO!

GOBRIDGE



План обсуждения

1. Когда, где и зачем появился Go
2. Рост адаптации языка в индустрии
3. 15 лет Go в этом году
4. Go как платформа
5. Минусы Go
6. Сравнение с C# (.NET) и Kotlin (JVM)
7. Go 2?
8. Почувствуй себя гошником
9. Ресурсы для самостоятельного ознакомления

package main UNREGISTERED

```go
package main

import (
    "fmt"
    "os"

    miniflux "miniflux.app/v2/client"
)

func main() {
    // Authentication with username/password:
    client := miniflux.New("https://api.example.org", "admin", "secret")

    // Authentication with an API Key:
    client := miniflux.New("https://api.example.org", "my-secret-token")

    // Fetch all feeds.
    feeds, err := client.Feeds()
    if err != nil {
        fmt.Println(err)
        return
    }
    fmt.Println(feeds)

    // Backup your feeds to an OPML file.
    opml, err := client.Export()
    if err != nil {
        fmt.Println(err)
        return
    }

    err = os.WriteFile("opml.xml", opml, 0644)
    if err != nil {
        fmt.Println(err)
        return
    }
}
```

untitled UNREGISTERED

```go
)

// User represents a user in the system.
type User struct {
    ID                    int64      `json:"id"`
    Username              string     `json:"username"`
    Password              string     `json:"password,omitempty"`
    IsAdmin               bool       `json:"is_admin"`
    Theme                 string     `json:"theme"`
    Language              string     `json:"language"`
    Timezone              string     `json:"timezone"`
    EntryDirection        string     `json:"entry_sorting_direction"`
    EntryOrder            string     `json:"entry_sorting_order"`
    Stylesheet            string     `json:"stylesheet"`
    CustomJS              string     `json:"custom_js"`
    GoogleID              string     `json:"google_id"`
    OpenIDConnectID       string     `json:"openid_connect_id"`
    EntriesPerPage        int        `json:"entries_per_page"`
    KeyboardShortcuts     bool       `json:"keyboard_shortcuts"`
    ShowReadingTime       bool       `json:"show_reading_time"`
    EntrySwipe            bool       `json:"entry_swipe"`
    GestureNav            string     `json:"gesture_nav"`
    LastLoginAt           *time.Time `json:"last_login_at"`
    DisplayMode           string     `json:"display_mode"`
    DefaultReadingSpeed   int        `json:"default_reading_speed"`
    CJKReadingSpeed       int        `json:"cjk_reading_speed"`
    DefaultHomePage       string     `json:"default_home_page"`
    CategoriesSortingOrder string     `json:"categories_sorting_order"`
    MarkReadOnView        bool       `json:"mark_read_on_view"`
    MediaPlaybackRate     float64    `json:"media_playback_rate"`
    BlockFilterEntryRules string     `json:"block_filter_entry_rules"`
    KeepFilterEntryRules  string     `json:"keep_filter_entry_rules"`
    ExternalFontHosts     string     `json:"external_font_hosts"`
}

func (u User) String() string {
    return fmt.Sprintf("#%d - %s (admin=%v)", u.ID, u.Username, u.IsAdmin)
}
```

Когда, где и зачем появился Go

Кто создал

Роберт Гризмер, Роб Пайк и Кен Томпсон в Google в 2007 году.

Зачем придумали

Хотели язык:

- Лёгкий и простой, без сложностей C++ и Java
- Быстрый – чтобы компиляция занимала секунды на больших проектах
- Удобный для работы с многопоточностью
- Безопасный (memory safety, concurrency bugs)
- Подходящий для разработки и поддержки сложных систем, в том числе в больших командах

Первые релизы

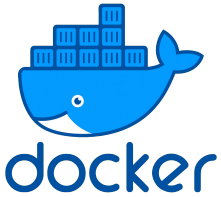
- Публичный анонс в 2009
- Версия 1.0 в 2012

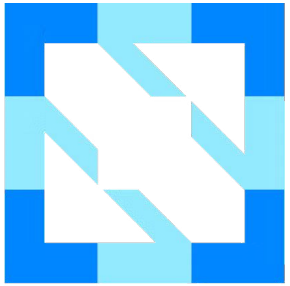
Проекты внутри Google

- Переписали dl.google.com с C++ – сервис загрузки Google Chrome, Android, Earth и прочего софта Google → значительно сократили latency и улучшили поддержку кодовой базы
- Переписали Vitess, БД на основе MySQL, с C++ и вынесли в OSS
- Команда браузера Chrome мигрировала некоторые инфраструктурные инструменты с C++ на Go

Рост адаптации языка в индустрии

Инфраструктурный софт





CLOUD NATIVE
COMPUTING FOUNDATION

Миграция legacy-сервисов и разработка новых

- **Dropbox** переписали движок синхронизации: Python → Go
- **Twitch** переписали часть сервисов: Python → Go
- **Uber** управление VM, инфраструктура и хранилища данных → Go
- **Microsoft Azure, AWS, DigitalOcean** – облачные решения
- **CloudFlare** – сетевая инфраструктура
- **PayPal** – несколько бэкенд-сервисов
- **BBC** – микро-сервисы на Go
- **SoundCloud** – бэкенд сервисы на Go
- **Monzo** – бэкенд-сервисы на Go
- **Тинькофф**, **Ozon, Яндекс, X5, etc.** – миграция с PHP, Ruby, Python, etc.

БД

- CockroachDB
- InfluxDB
- Victoria Metrics
- …

https://awesome-go.com/databases-implemented-in-go/

Прочий софт

- Hugo – статический генератор сайтов
- Caddy – веб-сервер с поддержкой Let’s Encrypt
- Minio – S3-совместимое хранилище
- Syncthing – синхронизация файлов
- Imageproxy – обработка изображений
- Ethereum – блокчейн

CLI, бэкенды self-hosted сервисов и т.д.

Чем еще пользуюсь:

- Miniflux (RSS)
- Readeck (Offline reader)

15 лет Go в этом году

Go 1.0 to 1.5 (2012-2015):

- Поддержка обратной совместимости в Go 1.x
- Компилятор переписан с C на Go
- Новый GC

Go 1.6 to 1.11 (2016-2018):

- Добавлена поддержка HTTP/2
- В стандартную библиотеку добавлен пакет context
- Модули (go mod) для управления зависимостями

Go 1.12 to 1.18 (2019-2022):

- Generics (major language feature)
- Улучшения в модулях
- Улучшения в обработке ошибок
- Вырос performance

Go 1.19 to 1.22 (2022-2024):

- Profile-guided optimization (PGO)
- Улучшения GC (еще меньше пауз)
- Улучшения в fuzzing
- Поддержка workspaces (окружений) для мульти-модульной разработки

Ресурсы для самостоятельного ознакомления

- Интервью Роберта Пайка про Go

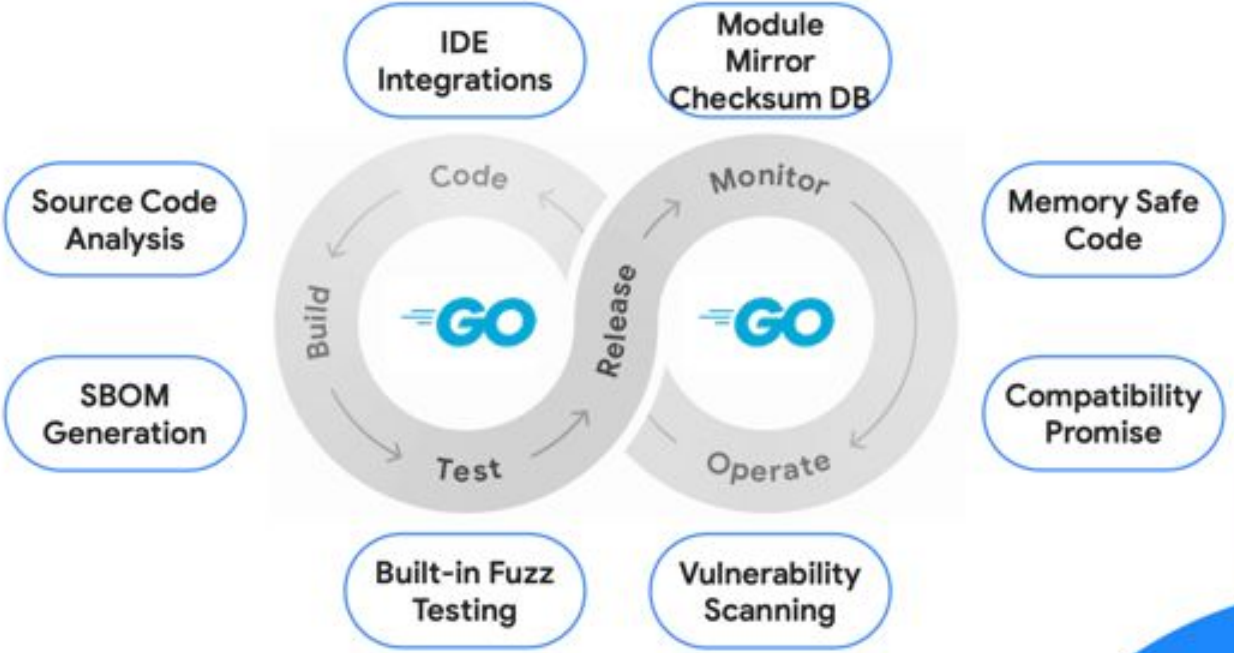
Go как платформа

Go's developer platform provides end-to-end solutions out of the box

Go's developer platform provides end-to-end solutions for developer velocity

- Конкурентное программирование: горутины и каналы
- Интеграция с IDE: Jetbrains' Goland, Go plugin для Visual Studio Code
- Управление зависимостями: встроенный тулинг для работы с модулями – `go get` или `go mod tidy` для установки зависимостей проекта
- Кросс-компиляция: например, на macOS можно сгенерировать бинарник для Windows, Linux и др. платформ
- Статические бинарники: компилируем под любую платформу → self-contained executable
- Встроенный дебаггер Delve
- Фреймворк для тестирования в стандартной библиотеке
- Встроенные инструменты форматирования и линтинга кода

Go's developer platform provides end-to-end solutions for security



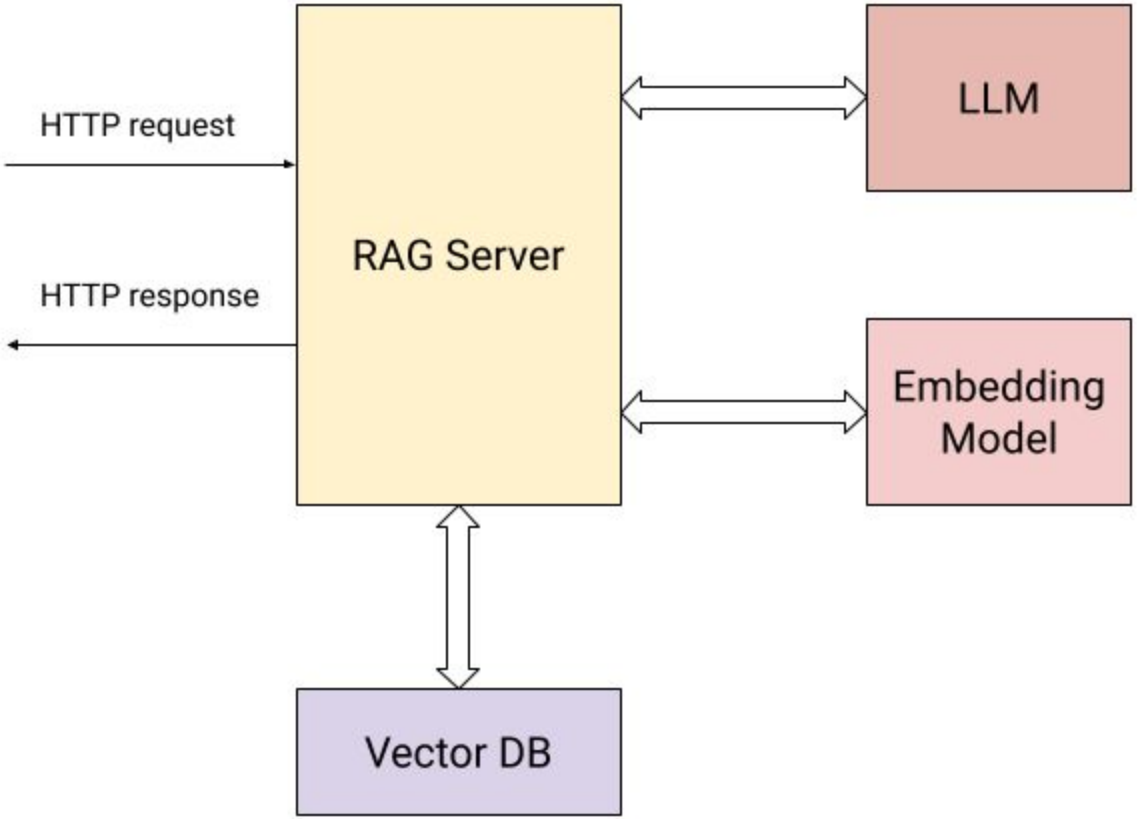
- Module Mirror Checksum DB: добавлено в 2019
- Memory Safe Code
- Compatibility Promise
- Vulnerability Scanning: govulncheck еще один инструмент из тулинга языка, который позволяет находить и исправлять уязвимости в проектах
- Built-in Fuzz Testing
- Генерация SBOM (Software Bill of Materials) – списка используемых компонент в проекте
- Анализ исходного кода – как коммерческие решения типа Sonar так и open-source типа golangci-lint

Go's developer platform provides end-to-end solutions for performance

Proprietary

- Отличная стандартная библиотека: сеть, маршалинг, тестирование, структуры данных и т.д.
- Встроенные средства профайлинга: `pprof`
- Runtime Tracing для понимания поведения приложений в runtime
- Self-tuning GC – один из самых современных GC
- Динамическое определение гонок состояний
- Profile-guided Optimization (PGO) – значительно сокращает потребление ресурсов

Разработка LLM-сервисов на Go



Минусы Go

«Go doesn’t really do anything new»

«Go isn’t well-designed from the ground up»

«Go is a regression from other modern programming languages»

- Система типов сильно уступает другим современным языкам
- Использование Interface могут часто приводит к ошибкам в runtime
- Слабая реализация дат и времени, некоторых других пакетов stdlib
- Statements over Expressions
- Философия обработки ошибок
- Код на Go часто выглядит скучным, повторяющимся, мало выразительным
- Отсутствие full-stack фреймворков для разработки монолитных систем

untitled UNREGISTERED

```go
// start the heartbeat by setting up necesssary handlers and looping by sending "ping"
// message every "period" until the "closer" channel is closed.
func (h *heartbeat) start() {
    // Loop to continually send "ping" message through websocket connection every "period".
    t := time.NewTicker(h.period)
    defer t.Stop()
    for {
        select {
        case <-h.closer:
            klog.V(8).Infof("closed channel--returning")
            return
        case <-t.C:
            // "WriteControl" does not need to be protected by a mutex. According to
            // gorilla/websockets library docs: "The Close and WriteControl methods can
            // be called concurrently with all other methods."
            if err := h.conn.WriteControl(gwebsocket.PingMessage, h.message, time.Now().Add(pingReadDeadline)); err == nil {
                klog.V(8).Infof("Websocket Ping succeeeded")
            } else {
                klog.Errorf("Websocket Ping failed: %v", err)
                if errors.Is(err, gwebsocket.ErrCloseSent) {
                    // we continue because c.conn.CloseChan will manage closing the connection already
                    continue
                } else if e, ok := err.(net.Error); ok && e.Timeout() {
                    // Continue, in case this is a transient failure.
                    // c.conn.CloseChan above will tell us when the connection is
                    // actually closed.
                    // If Temporary function hadn't been deprecated, we would have used it.
                    // But most of temporary errors are timeout errors anyway.
                    continue
                }
                return
            }
        }
    }
}
```

Сравнение с Kotlin (JVM) и C# (.NET)

Задача

Конкурентно обойти список ссылок, скачать контент и сохранить его в каталоге “downloads”

Реализации

- Go (113)
- C# (87)
- Kotlin (68)

Go 2?

Go 2, here we come! (2018)

Backward Compatibility, Go 1.21, and Go 2

Почувствуй себя гошником

Хочу пощупать прям сейчас → The Go Playground

Хочу по-серьезному в редакторе:

1. Устанавливаем Go
2. Настраиваем Visual Studio Code
3. Клонируем https://github.com/golang/example
4. Заходим в папочку example → открываем ее VS Code → читаем README → запускаем выбранный пример → смотрим результат → меняем что-нибудь в коде 🔁

Ресурсы для самостоятельного изучения

- Книга “Язык программирования Go”
- Официальный сайт
- How I Write HTTP Web Services after Eight Years - Mat Ryer